PROGAMACIÓN DIDÁCTICA DEL MÓDULO PROGRAMACIÓN (MODALIDAD SEMIPRESENCIAL)

Índice:

1 DATOS IDENTIFICATIVOS, MARCO NORMATIVO Y CONTEXTUALIZACIÓN. 1
2 RELACIÓN ENTRE ESTÁNDARES DE COMPETENCIA Y LOS MÓDULOS DEL CICLO FORMATIVO. 3
3 CONTRIBUCIÓN DE LOS RA A LAS COMPETENCIAS GENERALES. 3
4 ESQUEMA GENERAL Y SECUENCIACIÓN DE LAS UNIDADES DE PROGRAMACIÓN. 5
5 METODOLOGÍA DEL PROCESO DE ENSEÑANZA-APRENDIZAJE. 21
6 RECURSOS. 22
7 USO DE ESPACIOS Y EQUIPAMIENTOS. 23
8 MEDIDAS DE ATENCIÓN A LA DIVERSIDAD. 23
9 EVALUACIÓN DEL APRENDIZAJE. 24
10 ACTIVIDADES COMPLEMENTARIAS Y EXTRAESCOLARES 26
11 CRITERIOS Y PROCEDIMIENTOS PARA LA EVALUACIÓN DEL DESARROLLO DE LA PROGRAMACIÓN Y DE LA PRÁCTICA DOCENTE, ASÍ COMO LOS CRITERIOS DE CALIFICACIÓN. 26
12 PLAN DE RECUPERACIÓN. 28
13 PLAN DE FOMENTO DE LA LECTURA. 28
14 PROYECTO INTERMODULAR. 28

1 DATOS IDENTIFICATIVOS, MARCO NORMATIVO Y CONTEXTUALIZACIÓN.

1.1 DATOS IDENTIFICATIVOS.

El módulo profesional “Programación” pertenece al ciclo formativo “Desarrollo de Aplicaciones Multiplataforma” de Formación Profesional de Grado Superior de la familia profesional “Informática y Comunicaciones”. Sus datos identificativos:

Código: 0485
Denominación: Programación
Equivalencia en créditos ECTS: 14
Duración: 256 horas (8 sesiones semanales):

- Tutorías individuales: 2 matinales y 2 vespertinas.
- Tutorías colectivas: 2 matinales y 2 vespertinas.

Familia profesional: Informática y Comunicaciones
Ciclo formativo de grado superior: Desarrollo de Aplicaciones Multiplataforma

Centro: IES. Serra Perenxisa (Torrent, Valencia)

Profesor: Carlos Tarazona Tárrega

Curso: 2026 – 2027

1.2 MARCO NORMATIVO.

Se le aplica la normativa recogida en el PCFF de DAM y que se enumera a continuación:

- RD 659/2023, de 18 de julio
- LO 3/2022
- RD 500/2024, de 21 de mayo
- Propuesta Decreto Currículum GVA

Título

- RD 450/2010, de 16 de abril.
- RD 405/2023, de 29 de mayo
- O 58/2012, de 5 de septiembre

Dual

- Decreto 74/2013, de 14 de junio
- O 5/2022, de 15 de febrero

Evaluación en ciclos Formativos

- Orden 8/2025 del 22 de abril

Tutoría y orientación profesional

- Decreto 72/21.
- Orden 10/2023

1.3 CONTEXTUALIZACIÓN.

El módulo de programación se imparte en el primer curso del CFGS de Técnico Superior en Desarrollo de Aplicaciones Multiplataforma (DAM) en concreto en la comarca de l'Horta Sud, donde se sitúa la ciudad de Torrent y el IES Serra Perenxisa que presenta un entorno económico dinámico, con una importante presencia de pequeñas y medianas empresas en sectores como el comercio, la hostelería, la administración pública, la logística, el transporte y los servicios sociales. Estas actividades requieren cada vez más soluciones digitales eficientes, seguras y adaptadas a diferentes dispositivos y plataformas, lo que encaja plenamente con las competencias desarrolladas en el ciclo de DAM.

Torrent, como uno de los municipios más poblados de la provincia de Valencia, cuenta con una red consolidada de infraestructuras, centros tecnológicos cercanos y acceso a polos de innovación como el Parc Tecnològic de Paterna o el ecosistema emprendedor de la ciudad de Valencia. Esto permite que el alumnado del IES Serra Perenxisa pueda establecer vínculos con empresas del sector TIC de la zona, realizar prácticas formativas (FCT) de

calidad y acceder a oportunidades de empleo local sin necesidad de desplazarse a grandes distancias.

Además, el centro se encuentra comprometido con los principios de innovación educativa, digitalización y sostenibilidad, lo que refuerza la integración de las competencias del título en un marco pedagógico actualizado y alineado con los retos del siglo XXI. Se promueve también el desarrollo de proyectos reales, el trabajo colaborativo, el uso de herramientas de software libre y la incorporación de metodologías activas que preparan al alumnado para entornos laborales ágiles y en constante evolución.

Desde el punto de vista cultural y lingüístico, el IES Serra Perenxisa fomenta el uso tanto del castellano como del valenciano, reflejo de la diversidad lingüística y cultural de la Comunidad Valenciana. Esta realidad también se traslada al ámbito del desarrollo de software, promoviendo la creación de aplicaciones accesibles, inclusivas y adaptadas al contexto sociocultural de la ciudadanía.

La procedencia de los alumnos de este ciclo, proporcionalmente, es muy abundante de los municipios cercanos. Suelen ser personas en edad laboral, muchas veces con trabajo que desean especializarse en un nuevo sector, actualizar sus conocimientos o perfeccionarlos.

2 RELACIÓN ENTRE ESTÁNDARES DE COMPETENCIA Y LOS MÓDULOS DEL CICLO FORMATIVO.

Tal y como recoge el PCCF de DAM se mantienen las relaciones establecidas en las recomendaciones pedagógicas del título. A continuación, se presenta la correspondencia entre los estándares de competencia profesional del título de DAM (según RD 450/2010, de 16 de abril, y el Decreto 405/2023, de 29 de mayo, del Consell) y los módulos profesionales del ciclo, destacando el papel del módulo Programación en el desarrollo de cada uno.

Estándar de competencia profesional	*Módulos del ciclo implicados*	*Contribución del módulo Programación*
UC0223_3: Desarrollar elementos software en el entorno de desarrollo integrado, utilizando lenguajes y librerías estándares.	Programación Entornos de desarrollo Bases de datos	Es el módulo principal donde se adquieren las competencias de codificación estructurada y orientada a objetos, manejo de librerías y depuración de código.
UC0226_3: Desarrollar aplicaciones informáticas sobre bases de datos relacionales.	Programación Bases de datos Acceso a datos	Introduce la conexión entre programas y bases de datos mediante JDBC o APIs equivalentes, preparando el terreno para el módulo Acceso

		a datos.
UC0496_3: Participar en el desarrollo de aplicaciones multiplataforma utilizando entornos y herramientas de desarrollo específicos.	Programación, Entornos de desarrollo, Programación multimedia y dispositivos móviles	Fomenta las bases de la programación multiplataforma (Java, C#, Kotlin...), la portabilidad y la modularidad del código.
UC0224_3: Diseñar la estructura y funcionalidad de aplicaciones informáticas.	Sistemas informáticos, Programación, Lenguajes de marcas	Proporciona los fundamentos lógicos y algorítmicos necesarios para convertir los diseños funcionales en código operativo.
UC0227_3: Realizar pruebas y depurar los programas desarrollados, verificando su funcionamiento.	Programación, Entornos de desarrollo	Se trabajan técnicas de depuración, pruebas unitarias y gestión de errores.
UC0225_3: Elaborar documentación técnica y de usuario de la aplicación.	Programación, Entornos de desarrollo, Proyecto final	Introduce la documentación técnica y el uso de comentarios, convenciones de estilo y generación automática de documentación.

El módulo de **Programación**:

- Constituye el **eje vertebrador del primer curso**, donde se desarrollan las bases de la lógica, la codificación estructurada y la orientación a objetos.
- Su dominio es **imprescindible para el aprovechamiento de los módulos de segundo curso**, como *Acceso a Datos*, *Desarrollo de Interfaces* y *Programación de Servicios y Procesos*.
- Favorece competencias transversales: resolución de problemas, trabajo en equipo, gestión de versiones y buenas prácticas de documentación y mantenimiento.

3 CONTRIBUCIÓN DE LOS RA A LAS COMPETENCIAS GENERALES.

El módulo de **Programación** contribuye de forma esencial al desarrollo de las **competencias generales del título**, especialmente en lo relativo al diseño, construcción y mantenimiento de aplicaciones informáticas multiplataforma.

A continuación, se describe la relación entre cada **Resultado de Aprendizaje (RA)** del módulo y las **competencias generales (CG)** del ciclo de DAM.

RA	***Contribución a Competencias Generales del título***
RA1. Reconoce los elementos de un programa informático, describiendo su función y las fases de desarrollo del software.	Contribuye a las CG1 y CG3, desarrollando la comprensión del proceso de construcción de software y la planificación estructurada de proyectos informáticos. Facilita el trabajo coordinado con otros roles del desarrollo (análisis, diseño, pruebas).
RA2. Utiliza estructuras de control y datos, aplicando los principios de programación estructurada.	Potencia las CG2 y CG6 al favorecer la resolución lógica de problemas, el pensamiento algorítmico y la capacidad de aplicar estructuras eficientes en contextos reales.
RA3. Desarrolla programas estructurados, analizando y aplicando los principios de la programación modular.	Relacionado con las CG1, CG4 y CG6, fomenta la capacidad de diseñar código reutilizable y mantener la coherencia entre módulos de software, facilitando el trabajo colaborativo y el mantenimiento de aplicaciones complejas.
RA4. Desarrolla programas utilizando objetos y clases, aplicando los fundamentos de la programación orientada a objetos.	Aporta a las CG1, CG2 y CG5 al consolidar la base técnica sobre la que se construyen las aplicaciones orientadas a objetos. Favorece la abstracción, encapsulación y reutilización de componentes.
RA5. Realiza pruebas y depuración de programas, aplicando técnicas y herramientas específicas.	Fortalece las CG3, CG6 y CG7 al introducir prácticas de aseguramiento de la calidad, uso de entornos de depuración y control de errores, y adopción de metodologías de pruebas sistemáticas.
RA6. Documenta programas y desarrolla propuestas de mejora, siguiendo normas de calidad y usabilidad.	Contribuye directamente a las CG4, CG7 y CG8 al promover la comunicación técnica, la claridad en la documentación, la reflexión crítica y la mejora continua de los productos de software.

- El módulo de **Programación**:

- Desarrolla las **competencias técnicas fundamentales** sobre las que se construyen las aplicaciones multiplataforma del segundo curso.
- Favorece **competencias transversales** como el pensamiento lógico, la planificación, la comunicación técnica y la colaboración.
- Introduce al alumnado en **prácticas de calidad del software**, trabajo con estándares y metodologías ágiles básicas.
- Constituye la base sobre la que se articula la **competencia general del ciclo**:
- "Desarrollar, implantar y mantener aplicaciones informáticas multiplataforma, utilizando tecnologías y entornos de desarrollo específicos, garantizando el acceso a los datos de forma segura y cumpliendo los criterios de 'usabilidad' y calidad exigidos en los estándares establecidos."

4 ESQUEMA GENERAL Y SECUENCIACIÓN DE LAS UNIDADES DE PROGRAMACIÓN.

Las líneas de actuación en el proceso de enseñanza-aprendizaje que permiten alcanzar los objetivos del módulo versarán sobre:

– La interpretación y aplicación de los principios de la programación orientada a objetos.
– La evaluación, selección y utilización de herramientas y lenguajes de programación orientados a objetos.
– La utilización de las características específicas de lenguajes y entornos de programación en el desarrollo de aplicaciones informáticas.
– La identificación de las funcionalidades aportadas por los sistemas gestores de bases de datos y su incorporación a las aplicaciones desarrolladas.
– La prueba, depuración y documentación de las aplicaciones desarrolladas.

Los resultados de aprendizaje y sus criterios de evaluación en que se concreta el proceso de enseñanza-aprendizaje y que van a ser objeto de evaluación son:

RA 1. Reconoce la estructura de un programa informático, identificando y relacionando los elementos propios del lenguaje de programación utilizado. Criterios de evaluación:

a) Se han identificado los bloques que componen la estructura de un programa informático.
b) Se han creado proyectos de desarrollo de aplicaciones.
c) Se han utilizado entornos integrados de desarrollo.
d) Se han identificado los distintos tipos de variables y la utilidad específica de cada uno.
e) Se ha modificado el código de un programa para crear y utilizar variables.
f) Se han creado y utilizado constantes y literales.
g) Se han clasificado, reconocido y utilizado en expresiones los operadores del lenguaje.
h) Se ha comprobado el funcionamiento de las conversiones de tipo explícitas e implícitas.

i) Se han introducido comentarios en el código.

RA 2. Escribe y prueba programas sencillos, reconociendo y aplicando los fundamentos de la programación orientada a objetos. Criterios de evaluación:

a) Se han identificado los fundamentos de la programación orientada a objetos.
b) Se han escrito programas simples.
c) Se han instanciado objetos a partir de clases predefinidas.
d) Se han utilizado métodos y propiedades de los objetos.
e) Se han escrito llamadas a métodos estáticos.
f) Se han utilizado parámetros en la llamada a métodos.
g) Se han incorporado y utilizado librerías de objetos.
h) Se han utilizado constructores.
i) Se ha utilizado el entorno integrado de desarrollo en la creación y compilación de programas simples.

RA 3. Escribe y depura código, analizando y utilizando las estructuras de control del lenguaje. Criterios de evaluación:

a) Se ha escrito y probado código que haga uso de estructuras de selección.
b) Se han utilizado estructuras de repetición.
c) Se han reconocido las posibilidades de las sentencias de salto.
d) Se ha escrito código utilizando control de excepciones.
e) Se han creado programas ejecutables utilizando diferentes estructuras de control.
f) Se han probado y depurado los programas.
g) Se ha comentado y documentado el código.

RA 4. Desarrolla programas organizados en clases analizando y aplicando los principios de la programación orientada a objetos. Criterios de evaluación:

a) Se ha reconocido la sintaxis, estructura y componentes típicos de una clase.
b) Se han definido clases.
c) Se han definido propiedades y métodos.
d) Se han creado constructores.
e) Se han desarrollado programas que instancien y utilicen objetos de las clases creadas anteriormente.
f) Se han utilizado mecanismos para controlar la visibilidad de las clases y de sus miembros.
g) Se han definido y utilizado clases heredadas.
h) Se han creado y utilizado métodos estáticos.
i) Se han definido y utilizado interfaces.
j) Se han creado y utilizado conjuntos y librerías de clases.

RA 5. Realiza operaciones de entrada y salida de información, utilizando procedimientos específicos del lenguaje y librerías de clases. Criterios de evaluación:

a) Se ha utilizado la consola para realizar operaciones de entrada y salida de información.
b) Se han aplicado formatos en la visualización de la información.

c) Se han reconocido las posibilidades de entrada / salida del lenguaje y las librerías asociadas.
d) Se han utilizado ficheros para almacenar y recuperar información.
e) Se han creado programas que utilicen diversos métodos de acceso al contenido de los ficheros.
f) Se han utilizado las herramientas del entorno de desarrollo para crear interfaces gráficos de usuario simples.
g) Se han programado controladores de eventos.
h) Se han escrito programas que utilicen interfaces gráficos para la entrada y salida de información.

RA 6. Escribe programas que manipulen información seleccionando y utilizando tipos avanzados de datos. Criterios de evaluación:

a) Se han escrito programas que utilicen arrays.
b) Se han reconocido las librerías de clases relacionadas con tipos de datos avanzados.
c) Se han utilizado listas para almacenar y procesar información.
d) Se han utilizado iteradores para recorrer los elementos de las listas.
e) Se han reconocido las características y ventajas de cada una de las colecciones de datos disponibles.
f) Se han creado clases y métodos genéricos.
g) Se han utilizado expresiones regulares en la búsqueda de patrones en cadenas de texto.
h) Se han identificado las clases relacionadas con el tratamiento de documentos escritos en diferentes lenguajes de intercambio de datos.
i) Se han realizado programas que realicen manipulaciones sobre documentos escritos en diferentes lenguajes de intercambio de datos.
j) Se han utilizado operaciones agregadas para el manejo de información almacenada en colecciones.

RA 7. Desarrolla programas aplicando características avanzadas de los lenguajes orientados a objetos y del entorno de programación. Criterios de evaluación:

a) Se han identificado los conceptos de herencia, superclase y subclase.
b) Se han utilizado modificadores para bloquear y forzar la herencia de clases y métodos.
c) Se ha reconocido la incidencia de los constructores en la herencia.
d) Se han creado clases heredadas que sobrescriban la implementación de métodos de la superclase.
e) Se han diseñado y aplicado jerarquías de clases.
f) Se han probado y depurado las jerarquías de clases.
g) Se han realizado programas que implementen y utilicen jerarquías de clases.
h) Se ha comentado y documentado el código.
i) Se han identificado y evaluado los escenarios de uso de interfaces.
j) Se han identificado y evaluado los escenarios de utilización de la herencia y la composición.

RA 8. Utiliza bases de datos orientadas a objetos, analizando sus características y aplicando técnicas para mantener la persistencia de la información. Criterios de evaluación:

a) Se han identificado las características de las bases de datos orientadas a objetos.
b) Se ha analizado su aplicación en el desarrollo de aplicaciones mediante lenguajes orientados a objetos.
c) Se han instalado sistemas gestores de bases de datos orientados a objetos.
d) Se han clasificado y analizado los distintos métodos soportados por los sistemas gestores para la gestión de la información almacenada.
e) Se han creado bases de datos y las estructuras necesarias para el almacenamiento de objetos.
f) Se han programado aplicaciones que almacenen objetos en las bases de datos creadas.
g) Se han realizado programas para recuperar, actualizar y eliminar objetos de las bases de datos.
h) Se han realizado programas para almacenar y gestionar tipos de datos estructurados, compuestos y relacionados.

RA 9. Gestiona información almacenada en bases de datos relacionales manteniendo la integridad y consistencia de los datos. Criterios de evaluación:

a) Se han identificado las características y métodos de acceso a sistemas gestores de bases de datos relacionales.
b) Se han programado conexiones con bases de datos.
c) Se ha escrito código para almacenar información en bases de datos.
d) Se han creado programas para recuperar y mostrar información almacenada en bases de datos.
e) Se han efectuado borrados y modificaciones sobre la información almacenada.
f) Se han creado aplicaciones que ejecuten consultas sobre bases de datos.
g) Se han creado aplicaciones para posibilitar la gestión de información presente en bases de datos.

De todos los criterios de evaluación asociados a los resultados de aprendizaje, aproximadamente entre el 10% y el 15% se aplican durante la realización de las prácticas en la empresa. La siguiente tabla establece los criterios de cada resultado de aprendizaje que se evalúan en la empresa:

Resultado de Aprendizaje	Criterio de Evaluación Evaluado por la Empresa
RA 1. Reconoce la estructura de un programa informático, identificando y relacionando los elementos propios del lenguaje de	i) Se han introducido comentarios en el código.

programación utilizado.	
RA 2. Escribe y prueba programas sencillos, reconociendo y aplicando los fundamentos de la programación orientada a objetos.	i) Se ha utilizado el entorno integrado de desarrollo en la creación y compilación de programas simples.
RA 3. Escribe y depura código, analizando y utilizando las estructuras de control del lenguaje.	f) Se han probado y depurado los programas.
RA 4. Desarrolla programas organizados en clases analizando y aplicando los principios de la programación orientada a objetos.	j) Se han creado y utilizado conjuntos y librerías de clases.
RA 5. Realiza operaciones de entrada y salida de información, utilizando procedimientos específicos del lenguaje y librerías de clases.	a) Se ha utilizado la consola para realizar operaciones de entrada y salida de información.
RA 6. Escribe programas que manipulen información seleccionando y utilizando tipos avanzados de datos.	i) Se han realizado programas que realicen manipulaciones sobre documentos XML.
RA 7. Desarrolla programas aplicando características avanzadas de los lenguajes orientados a objetos y del entorno de programación.	f) Se ha comentado y documentado el código.
RA 8. Utiliza bases de datos orientadas a objetos, analizando sus características y aplicando técnicas para mantener la persistencia de la información.	a) Se han identificado las características de las bases de datos orientadas a objetos.
RA 9. Gestiona información almacenada en bases de datos relacionales manteniendo la integridad y consistencia de los datos.	a) Se han identificado las características y métodos de acceso a sistemas gestores de bases de datos relacionales.

Además, hay que mencionar tal y como aparece indicado también en el apartado 14, el RA 2 se trabajará también en el proyecto intermodular.

Contenidos Mínimos.

Los contenidos básicos que propone el Real Decreto son los siguientes:

- Identificación de los elementos de un programa informático: Estructura y bloques fundamentales. Variables. Tipos de datos. Literales. Constantes. Operadores y expresiones. Conversiones de tipo. Comentarios.
- Utilización de objetos: Características de los objetos. Instanciación de objetos. Utilización de métodos. Utilización de propiedades. Utilización de métodos estáticos. Constructores. Destrucción de objetos y liberación de memoria.
- Uso de estructuras de control: Estructuras de selección. Estructuras de repetición. Estructuras de salto. Control de excepciones.
- Desarrollo de clases: Concepto de clase. Estructura y miembros de una clase. Creación de atributos. Creación de métodos. Creación de constructores. Utilización de clases y objetos. Utilización de clases heredadas.
- Lectura y escritura de información: Tipos de flujos. Flujos de bytes y de caracteres. Clases relativas a flujos. Utilización de flujos. Entrada desde teclado. Salida a pantalla. Ficheros de datos. Registros. Apertura y cierre de ficheros. Modos de acceso. Escritura y lectura de información en ficheros. Utilización de los sistemas de ficheros. Creación y eliminación de ficheros y directorios.
- Aplicación de las estructuras de almacenamiento: Estructuras. Creación de arrays. Arrays multidimensionales. Cadenas de caracteres. Listas.
- Utilización avanzada de clases: Composición de clases. Herencia. Superclases y subclases. y métodos abstractos y finales. Sobrescritura de métodos. Constructores y herencia. Interfaces. Concepto de evento. Creación de controladores de eventos.
- Mantenimiento de la persistencia de los objetos: Bases de datos orientadas a objetos. Características de las bases de datos orientadas a objetos. Instalación del gestor de bases de datos. Creación de bases de datos. Mecanismos de consulta. El lenguaje de consultas: sintaxis, expresiones, operadores. Recuperación, modificación y borrado de información.
- Tipos de datos objeto: atributos y métodos. Tipos de datos colección.
- Gestión de bases de datos relacionales: Establecimiento de conexiones. Recuperación de información. Manipulación de la información. Ejecución de consultas sobre la base de datos.

Concreción

Estos contenidos básicos propuestos por el Real Decreto se redistribuirán, de modo que su aprendizaje sea progresivo y permita la realización de actividades prácticas tan pronto como sea posible. Los contenidos a impartir se organizan en una serie de unidades de trabajo agrupadas bajo los siguientes bloques conceptuales que desarrollan diferentes áreas:

1. Introducción a la programación con Java.

2. Programación orientada a objetos.

3. Persistencia: Acceso a ficheros, persistencia y a acceso a bases de datos.

La finalidad de los objetivos que se persiguen con cada bloque:

Bloque 1: Programación con Java.

El objetivo principal es que el alumno adquiera los conceptos básicos de la programación, mediante la adquisición de métodos y técnicas para la resolución de problemas, así como las formas descriptoras en forma de pseudocódigo para la resolución de algoritmos que resuelvan esos problemas planteados, siguiendo un diseño modular y estructurado. Se comenzará relacionando la sintaxis de las estructuras de datos simples, y las sentencias elementales de los lenguajes y el proceso de construcción de los programas junto con las herramientas que se utilizan (compiladores, intérpretes, mixtos, ...) y reconozcan las características y diferencias de cada lenguaje de los que comentaremos los principales paradigmas que se utilizan. Posteriormente se comienza a trabajar en lenguaje Java, la instalación del entorno de desarrollo, los elementos básicos del lenguaje, las sentencias de control que permiten utilizar programación estructurada, las clases de uso habitual, estructuras de datos arrays, programación modular, etc.

Bloque 2: Programación orientada a objetos.

Una vez que el alumno se desenvuelva en la programación estructurada, se introducirá en la programación orientada a objetos, usando técnicas de diseño más naturales para descomponer problemas de mayor tamaño. Dominar el uso de clases, Objetos y métodos ayudarán en esta tarea junto la sobrecarga y sobrescritura de métodos y los modificadores de acceso. Más adelante profundizamos en las interfaces, la herencia, el polimorfismo, interfaces, clases abstractas, clases anidadas, expresiones lambda y su uso en la programación funcional.

Una vez vista la parte avanzada de orientación a objetos, pasamos a su aplicación en el uso de: colecciones de datos, uso de aplicaciones GUI, manipulación de datos en forma de streams.

Bloque 3: Ficheros, persistencia y a acceso a bases de datos.

Como distintas alternativas para que la información quede almacenada de forma permanente, se tratará el acceso a ficheros de datos mediante streams, manipulación de ficheros y carpetas, la persistencia de objetos y el acceso a bases de datos relacionales.

Elementos curriculares de cada unidad didáctica:

UNIDAD 1. INTRODUCCIÓN A LA PROGRAMACIÓN.	
Horas:	8
Conceptos:	Evolución y clasificación de lenguajes: bajo/alto nivel, tipados/flexibles, ensambladores, compiladores, intérpretes, Pseudocódigo, máquinas virtuales y bytecodes, ficheros fuente, ficheros compilados, ficheros ejecutables, Algoritmos, datos, sentencias, pseudocódigo, ordinogramas,

	trazado/depuración de un algoritmo/programa, subalgoritmos, tipo de paso de parámetros, programación desestructurada, estructurada, modular, orientada a objetos, funcional.
Procedimientos:	Ventajas e inconvenientes de lenguajes compilados contra interpretados y mixtos. Uso de pseudocódigo para expresar pequeños algoritmos y aplicar conceptos de programación: uso de sentencias if-else, for, while, repeat-until, do-while, funciones y procedimientos, paso de parámetros por valor y por referencia, uso de operadores aritméticos, su precedencia, la escritura de expresiones aritméticas, igual que las lógicas, conectores, ...
Actividades de enseñanza-aprendizaje:	Análisis de algoritmos, Identificar la programación estructurada, ejemplos de eliminación de saltos incondicionales en algoritmos, implementar algoritmos en un intérprete de psueudocódigo llamado pseInt, examinar equivalencia de unos bucles con otros, resolver problemas de manera iterativa y usando recursividad, definir datos variables, etc.
Contenidos:	1. INTRODUCCIÓN. 2. ALGORITMOS. 4. LENGUAJES DE PROGRAMACIÓN. 5. CONSTRUIR PROGRAMAS. 6. CONOCER EL LENGUAJE DE PROGRAMACIÓN JAVA. 7. EJERCICIOS.
Competencias:	UC0404_3
Objetivos del ciclo:	e), j), q)
Resultados de aprendizaje:	1
Criterios de evaluación:	1.a) 1.e) 1.f) 1.i) Distinguir tipos de lenguajes de programación y sus características. Expresar y entender algoritmos en pseudocódigo. Diferenciar y aprovechar elementos de un programa. Usar comentarios en el código.

UNIDAD 2. PRIMEROS PASOS CON JAVA.	
Horas:	24
Conceptos:	Datos constantes, variables, tipos de datos, identificadores, estructura de un programa, escritura de literales, operadores aritméticos y lógicos, la clase System, Math, String, clases para fechas y horas, BigInteger, sentencia de control condicional, enumeraciones. Secuencias de escape.
Procedimientos:	Instalar el entorno de desarrollo en diferentes plataformas. Elaborar programas sencillos (una clase y el código dentro de main). Declarar datos locales. Usar los operadores aritméticos y lógicos. Detectar la necesidad de casting de datos. Uso del condicional if y if-else, anidando y encadenando sentencias. Uso de la sentencia de selección múltiple switch. Importar clases. Utilizar métodos de clases prefabricadas: System y Math para realizar operaciones matemáticas. Manipular datos de tipo objeto como String, Date, Calendar, LocalDatetime y BigInteger. Definir contantes enumeradas. Generar salida de datos formateada con printf. Usar la clase Scanner para entrada de datos por consola.
Actividades de enseñanza-aprendizaje:	Instalación del entorno de desarrollo. Análisis de código que sirva de ejemplo de cada aspecto básico del lenguaje. Realización de programas que usen los elementos básicos del lenguaje. Ejercicios y prácticas.
Contenidos:	1. VERSIONES DE JAVA. 2. INSTALAR EL ENTORNO DE DESARROLLO. 3. PRIMEROS PROGRAMAS EN JAVA. 4. ELEMENTOS DE UN PROGRAMA. 5. CLASES Y OBJETOS PREFABRICADOS DE USO HABITUAL. 6. ENTRADA Y SALIDA BÁSICA DE DATOS. 7. SENTENCIAS DE DECISIÓN. 8. ENUMERACIONES. 9. EJERCICIOS.

Competencias:	UC0223_3 y UC0404_3
Objetivos del ciclo:	e) j) q)
Resultados de aprendizaje:	1 y 4
Criterios de evaluación:	1.a) 1.b) 1.d) 1.e) 1.f) 1.g) 1.h) 1.i) 1.j) Se han identificado los bloques que componen la estructura de un programa. Se han identificado los distintos tipos de variables y la utilidad específica de cada uno. Se ha modificado el código de un programa para crear y utilizar variables. Se han creado y utilizado constantes y literales. Se han clasificado, reconocido y utilizado en expresiones los operadores del lenguaje. Se ha comprobado el funcionamiento de las conversiones de tipo explícitas e implícitas. Se han introducido comentarios en el código. Se ha utilizado la consola para realizar operaciones de entrada y salida de información. Se han aplicado formatos en la visualización de la información. Se han reconocido las posibilidades de E/S del lenguaje.

UNIDAD 3. EXCEPCIONES, BUCLES, ARRAYS Y MÉTODOS.	
Horas:	28
Conceptos:	Tratamiento de excepciones, sentencia try-catch-finally y las clases Exception. Bucles en Java: while, do-while y for. Sentencias break y continue. Arrays unidimensionales y multidimensionales, simétricos y asimétricos. La clase Arrays. Descomposición modular. Definición, declaración, llamadas a métodos de tipo función. Tipos de métodos. Variables locales y globales. Paso de parámetros. Diferencias entre objetos mutables e inmutables. Recursividad. Control de precondiciones y postcondiciones: con excepciones (sentencias throw y throws) y aserciones. Firma de un método. Expresiones Regulares.
Procedimientos:	Realización de bloques repetitivos con while (si la condición de salida se comprueba al comienzo), do..while (si la condición se comprueba al final) y con for (cuando es un número de iteraciones conocido). Interrupción de bucles con

	break y con continue. Usar un IDE, aprender a definir proyectos, y uso del depurador. Interceptación de errores en tiempo de ejecución mediante excepciones. Establecer mecanismos sencillos de depuración de código mediante puntos de ruptura y análisis de expresiones y variables Conocer estructura y uso de una cadena de texto. Definición de arrays. Carga de datos en un array, en declaración o a posteriori. Acceso a los datos de un array. Manipular Strings y Arrays con bucles: algoritmos de búsqueda, procesado, ordenación...
Actividades de enseñanza-aprendizaje	Realización de bloques repetitivos con while, do..while y for). Seguimiento del valor de variables usando el depurador del entorno escogido. Almacenar datos repetitivos empleando arrays. Combinar el uso de estructuras y arrays para datos repetitivos formados por unidades más complejas. Procesado de cadenas de texto para obtener o modificar la información que contienen.
Contenidos:	1. EXCEPCIONES. 2. SENTENCIAS DE REPETICIÓN. 3. ARRAYS. 4. PROCESAR STRINGS CON BUCLES. 5. MÉTODOS. 6. EXPRESIONES REGULARES. 7. EJERCICIOS.
Competencias:	UC0223_3 y UC0404_3.
Objetivos del ciclo:	e) j) q) w)
Resultados de aprendizaje:	1, 2, 3, 6
Criterios de evaluación:	1.a) 1.b) 1.c) 2.a) 2.b) 2.c) 3.a) 3.c) 3.d) 3.e) 3.f) 3.g) 6.a) 6.f) Se han identificado los bloques que componen la estructura de un programa informático. Se han creado proyectos de desarrollo de aplicaciones. Se han utilizado entornos integrados de desarrollo. Se han escrito programas simples. Se ha escrito y probado código que haga uso de estructuras de selección. Se han utilizado estructuras de repetición. Se han

	reconocido las posibilidades de las sentencias de salto. Se ha escrito código utilizando control de excepciones. Se han creado programas ejecutables utilizando diferentes estructuras de control. Se han probado y depurado los programas. Se ha comentado y documentado el código. Se han escrito programas que utilicen arrays. Se han utilizado expresiones regulares en la búsqueda de patrones en cadenas de texto.

UNIDAD 4. CLASES Y OBJETOS.	
Horas:	28
Conceptos:	Sobrecarga y sobreescritura de métodos. La clase Object. Una clase canónica en Java: sobreescritura de toString, clone y equals. Definición de clases y de objetos en Java. Modificadores para métodos y atributos. Uso del modificador static. Propiedades. Nociones básicas de UML. Constructores y código inicializador de objetos. La palabra reservada this. Generadores automáticos de código.
Procedimientos:	Descomposición modular de problemas como método de obtención de funciones. Definición y uso de funciones. Especificación del valor de retorno de una función Uso de parámetros de funciones Creación de funciones recursivas Obtención de tipos básicos con TryParse para capturar excepciones Pruebas simples de programas mediante aserciones Identificar clases y objetos al implementar un problema. Elaboración de diagramas de clases para especificar la solución a un problema. Desarrollo de programas en que se apliquen los conceptos de la programación orientada a objetos.
Actividades de enseñanaza-aprendizaje:	Dado un problema, aplicar técnicas de descomposición modular para obtener las funciones más adecuadas para su resolución. Definición y uso de funciones, que devuelvan un valor o no (procedimientos). Definición de variables locales y globales. Uso de parámetros de funciones, por valor y por referencia. Creación de funciones recursivas. Obtención de distintos tipos básicos capturando excepciones en una sola línea con try-catch, try con recursos, try-finally o delegar su tratamiento con trhows. Lanzar excepciones con throw. Implementar métodos con parámetros

	variables, objetos mutables e inmutables. Comprobar precondiciones en la llamada y usar clases de excepciones. Identificar las clases y objetos existentes en un problema real. Elaborar diagramas de clases para especificar la solución a un problema. Implementar las clases aplicando los conceptos de la programación orientada a objetos. Rediseñar programas realizados anteriormente. Crear constructores que se apoyen en otros constructores. Crear arrays de objetos.
Contenidos:	1. INTRODUCCIÓN. 2. MÁS SOBRE MÉTODOS: sobrecarga, sobrescritura, recursividad, …. 3. CLASES Y OBJETOS. 4. CONSTRUCTORES E INICIALIZACIÓN. 5. IMPLEMENTAR CLASES CORRECTAMENTE EN JAVA. 6. CLASES ESPECIALES: SINGLETON, INMUTABLES, ENUMERACIONES. 7. MÁS CLASES DE USO HABITUAL. 8. PAQUETES, MÓDULOS, APIS Y FRAMEWORKS. 9. EJERCICIOS.
Competencias:	UC0404_3 y UC0227_3
Objetivos del ciclo:	e), j), q)
Resultados de aprendizaje	1, 2, 3 y 4
Criterios de evaluación:	1.a) 1.d) 1.f) 2.a) 2.b) 2.c) 2.d) 2.e) 2.f) 2.g) 2.h) 2.i) 2.j) 4.a) 4.b) 4.c) 4.d) 4.f) Se han identificado los bloques que componen la estructura de un programa informático. Se han clasificado, reconocido y utilizado en expresiones los operadores del lenguaje. Se han introducido comentarios en el código. Se han identificado los fundamentos de la programación orientada a objetos. Se han escrito programas simples. Se han instanciado objetos a partir de clases predefinidas. Se han utilizado métodos y propiedades de los objetos. Se han escrito llamadas a métodos estáticos. Se han utilizado parámetros en la llamada a métodos. Se han incorporado

	y utilizado librerías de objetos. Se han utilizado constructores. Se ha utilizado el entorno integrado de desarrollo en la creación y compilación de programas simples. Se ha reconocido la sintaxis, estructura y componentes típicos de una clase. Se han definido clases. Se han definido propiedades y métodos. Se han creado constructores. Se han desarrollado programas que instancien y utilicen objetos de las clases creadas anteriormente. Se han utilizado mecanismos para controlar la visibilidad de las clases y de sus miembros.

UNIDAD 5. PROGRAMACIÓN ORIENTADA A OBJETOS AVANZADA.	
Horas:	30
Conceptos:	Relaciones entre clases: clientelar, composición de clases, herencia. Superclases y subclases. Polimorfismo. Sobrecarga. Sobreescritura de métodos. La anotación @Overwrite. Métodos abstractos y finales. Constructores y herencia. La palabra clave super. Clases abstractas e interfaces. Clases anidadas: estáticas, no estáticas, locales y anónimas. Interfaces funcionales, expresiones lambda y anotación @FunctionalInterface. Mejoras de las interfaces en Java 8.
Procedimientos:	Descripción de un enunciado en términos de clases, objetos y relaciones de herencia y composición. Implementar código a partir de diagramas de clases. Corregir problemas de pérdida de modificadores o efectos colaterales al usar composición con objetos mutables usando varias técnicas: operador de copia, método clone, etc. Desarrollo de programas en que se apliquen los conceptos de la programación orientada a objetos. Resolución de problemas más complejos que incluyan herencia a varios niveles. Reescritura de métodos en subclases. Herencia en constructores. Uso de clases abstractas e interfaces para código incompleto o por definir. Utilizar clases anidadas para mejorar el código. Usar expresiones lambdas para reducir la cantidad de código y simplificar el paso de parámetros a métodos.
Actividades de enseñanza-aprendizaje.	Extraer las clases, objetos y relaciones de herencia existentes en un problema real. Interpretar diagramas de clases. Desarrollar programas aplicando los conceptos de la programación orientada a objetos. Rediseñar programas realizados anteriormente. Distinguir

	mediante casos prácticos cuándo utilizar clases normales, abstractas, interfaces, o clases anidadas. Crear subclases que redefinan algunos de los métodos existentes en las superclases. Crear constructores que se apoyen en otros constructores o en otros métodos de la superclase. Utilizar el polimorfismo con clases e interfaces.
Contenidos:	1. RELACIONES ENTRE CLASES. 2. COMPOSICIÓN DE CLASES. 3. HERENCIA Y JERARQUÍAS DE CLASES. 4. ANIDAMIENTO DE CLASES. 5. POLIMORFISMO. 6. CLASES ABSTRACTAS. 7. INTERFACES. 8. NUEVAS CARACTERÍSTICAS DE JAVA 8 Y EXPRESIONES LAMBDA. 9. EJERCICIOS.
Competencias:	UC0404_3 y UC0227_3
Objetivos del ciclo:	e) j) q) w)
Resultados de aprendizaje	4 y 7
Criterios de evaluación:	4.a) 4.b) 4.c) 4.d) 4.f) 4.g) 4.h) 4.i) 4.j) 7.a) 7.b) 7.c) 7.d) 7.e) 7.f) 7.g) 7.h) Se ha reconocido la sintaxis, estructura y componentes típicos de una clase. Se han definido clases. Se han definido propiedades y métodos. Se han creado constructores. Se han desarrollado programas que instancien y utilicen objetos de las clases creadas anteriormente. Se han utilizado mecanismos para controlar la visibilidad de las clases y de sus miembros. Se han definido y utilizado clases heredadas. Se han creado y utilizado métodos estáticos. Se han definido y utilizado interfaces. Se han creado y utilizado conjuntos y librerías de clases. Se han identificado los conceptos de herencia, superclase y subclase. Se han utilizado modificadores para bloquear y forzar la herencia de clases y métodos. Se ha reconocido la incidencia de los constructores en la herencia. Se han creado clases heredadas que sobrescriban la implementación de métodos de la superclase.

	Se han diseñado y aplicado jerarquías de clases. Se han probado y depurado las jerarquías de clases. Se han realizado programas que implementen y utilicen jerarquías de clases. Se ha comentado y documentado el código.

UNIDAD 6. GENÉRICOS Y COLECCIONES DE DATOS.	
Horas:	30
Conceptos:	Genéricos. Estructuras de datos prefabricadas: listas, pilas, colas, colas con prioridad, conjuntos y mapas. Ventajas e inconvenientes de las dos implementaciones: con arrays o con enlaces (Memoria dinámica). Otras colecciones no prefabricadas: árboles y grafos. Iteradores, bucle foreach y los métodos equals y hashCode. Influencia de las interfaces Comparable y Comparator para ordenar los elementos por algún criterio.
Procedimientos:	Practicar la definición de clases y métodos genéricos. Uso de colecciones para resolver de manera eficiente problemas reales. Creación de listas, pilas, colas, conjuntos y mapas usando las estructuras existentes en el lenguaje. Implementación de colecciones propias no proporcionadas por el lenguaje.
Actividades de enseñanza-aprendizaje:	Crear clases y métodos genéricas. Crear pilas usando las estructuras existentes en el lenguaje. Resolver problemas usando las estructuras existentes en el lenguaje. Crear y manipular los objetos almacenados en las colecciones y mapas. Implementar nuevas estructuras usando referencias.
Contenidos:	1. INTRODUCCIÓN. 2. GENÉRICOS. 3. ITERADORES Y COMPARADORES. 4. COLECCIONES PRECONSTRUIDAS. 5. CREA TUS PROPIAS COLECCIONES. 6. EJERCICIOS.
Competencias:	UC0404_3 y UC0227_3
Objetivos del ciclo:	e), j), q), w)

Resultados de aprendizaje	6 y 7
Criterios de evaluación:	6.a) 6.b) 6.d) 6.e) 6.f) 7.a), 7.b), 7.c) 7.d) 7.e) 7.f) Se han escrito programas que utilicen arrays. Se han reconocido las librerías de clases relacionadas con tipos de datos avanzados. Se han utilizado listas para almacenar y procesar información. Se han utilizado iteradores para recorrer los elementos de las listas. Se han reconocido las características y ventajas de cada una de las colecciones de datos disponibles.

UNIDAD 7. LA API STREAM	
Horas:	19
Conceptos:	Streams. Diferencias de trabajar con colecciones. Pipelines de sentencias. Enfoque funcional. Tipos de operaciones: intermedias y finales. Operaciones básicas: crear, filtrar, ordenar, procesar, transformar, convertir en colección o en valor. Operaciones avanzadas: agrupar, colectores personalizados. Introducción a la programación funcional y asíncrona.
Procedimientos:	Como resolver problemas sustituyendo las colecciones por streams. Generar streams a partir de colecciones, arrays, ficheros o listas de valores. Uso de las operaciones para resolver operaciones sencillas.
Actividades de enseñanza-aprendizaje:	Ejemplos de cómo resolver problemas sustituyendo colecciones por streams. Operaciones para generar streams a partir de colecciones, arrays, ficheros o listas de valores. Implementar programas que procesen datos.
Contenidos:	1. INTRODUCCIÓN. 2. EXPRESIONES LAMBDA 3. STREAMS. 4. COLECTORES PARA PROCESAMIENTOS COMPLEJOS. 5. PROGRAMACIÓN PARALELA Y ASÍNCRONA. 6. EJERCICIOS.
Competencias:	UC0404_3 y UC0227_3

Objetivos del ciclo:	e), j), q), w)
Resultados de aprendizaje	6, 7 y 8
Criterios de evaluación:	6.a) 6.b) 6.c) 6.d) 6.e) 6.f) 7.a) Se han escrito programas que utilicen arrays. Se han reconocido las librerías de clases relacionadas con tipos de datos avanzados. Se han utilizado listas para almacenar y procesar información. Se han utilizado iteradores para recorrer los elementos de las listas. Se han reconocido las características y ventajas de cada una de las colecciones de datos disponibles. Se han creado clases y métodos genéricos.

UNIDAD 8. FICHEROS Y STREAMS.	
Horas:	22
Conceptos:	Streams. Tipos: binarios y de texto; de entrada y de salida; de bajo o de alto nivel. Buffers para mejorar rendimiento: uso manual o automático. Operaciones de la API io para manipular ficheros y carpetas. Ficheros, url y uris. La interface nio2 y el concepto de paths. La interface Serializable y Externalizable. Persistencia de objetos en ficheros. Ficheros de acceso aleatorio. Procesamiento secuencial / aleatorio. Procesamiento de ficheros XML con distintos parsers. Diálogos gráficos para seleccionar ficheros para entrada y salida.
Procedimientos:	Detectar el stream más apropiado para realizar un trabajo con ficheros. Usar buffers para mejorar el rendimiento del procesamiento en ficheros grandes de forma manual y automática. Envolver unos streams en otros. Excepciones de entrada/salida y liberación de recursos en caso de fallo. Usar la sentencia try que libera automáticamente recursos. Manipular ficheros y carpetas: búsquedas, filtros, creación, borrado, propiedades, etc. Conversión entre ficheros, paths, url y uris. Persistir objetos usando la interface Serializable. Procesar ficheros de datos de manera secuencial y con acceso aleatorio.
Actividades de enseñanza-aprendizaje:	Programas de ejemplo del uso de los distintos streams para entrada/salida con ficheros. Mejora del rendimiento con el uso de buffers. Ejemplos de envoltura de unos streams en otros. Analizar las excepciones de entrada/salida y liberación de

	recursos en caso de fallo. Usar la sentencia try que libera automáticamente recursos. Ejemplos de cómo crear, modificar o borrar ficheros y carpetas. Realizar la conversión entre ficheros, paths, url y uris. Persistir objetos usando la interface Serializable. Procesar ficheros de datos de manera secuencial y con acceso aleatorio. Ejemplos del uso de diferentes parser para el procesamiento de ficheros XML.
Contenidos:	1. STREAMS (FLUJOS). 2. FICHEROS. 3. MANIPULAR XML. 4. MANIPULAR JSON. 5. LA API NIO2. 6. EJERCICIOS.
Competencias:	UC0223_3, UC0404_3 y UC0227_3
Objetivos del ciclo:	e) j) q) w)
Resultados de aprendizaje	2, 3, 4 y 5
Criterios de evaluación:	2.g) 3.g) 4.j) 5.a) 5.b) 5.c) 5.d) 5.e) 6.h) 6.i) Se han incorporado y utilizado librerías de objetos. Se ha comentado y documentado el código. Se han definido y utilizado interfaces. Se ha utilizado la consola para realizar operaciones de entrada y salida de información. Se han aplicado formatos en la visualización de la información. Se han reconocido las posibilidades de entrada / salida del lenguaje y las librerías asociadas. Se han utilizado ficheros para almacenar y recuperar información. Se han creado programas que utilicen diversos métodos de acceso al contenido de los ficheros. Se han utilizado las herramientas del entorno de desarrollo para crear interfaces gráficos de usuario simples. Se han identificado las clases relacionadas con el tratamiento de documentos XML. Se han realizado programas que realicen manipulaciones sobre documentos XML.

UNIDAD 9. APLICACIONES CONTROLADAS POR EVENTOS.

Horas:	22
Conceptos:	Las APIs awt y swing. Ventanas, bucle de bombeo de mensajes, Threads, eventos, componentes: paneles, layouts, labels, botones, listas, checks y radios, cajas de edición, image y graphics. El modelo vista-controlador y su uso en Swing.
Procedimientos:	Instalar y usar una herramienta de diseño de interfaces. Como se define una aplicación con GUI sencilla a mano y con la ayuda de una herramienta. Utilizar eventos de teclado, ratón y usuario. Como dibujar de manera básica en un componente de la GUI. Manipular imágenes y otros recursos.
Actividades de enseñanza-aprendizaje:	Implementar aplicaciones sencillas con GUI. Describir con ejemplos las propiedades y métodos de varios componentes GUI. Controlar la apariencia y el comportamiento de la interface con layouts o sin ellos. Utilizar eventos para realizar procesamiento en las aplicaciones. Realizar dibujos y animaciones en componentes: de manera manual, con doble buffer, carga, visualización y modificación de imágenes y otros recursos (ficheros, audio, etc.).
Contenidos:	1. INTRODUCCIÓN. 2. PRIMEROS PASOS. 3. EVENTOS. 4. ALGUNOS COMPONENTES DE LA GUI. 5. IMÁGENES Y RECURSOS. 6. DISEÑO DE INTERFACES DESDE EL IDE. 7. EJERCICIOS.
Competencias:	UC0227_3, UC0404_3, UC0223_3
Objetivos del ciclo:	e) j) q) w)
Resultados de aprendizaje	5
Criterios de evaluación:	5.f) 5.g) 5.h) Se han utilizado las herramientas del entorno de desarrollo para crear interfaces gráficos de usuario simples. Se han programado controladores de eventos. Se han escrito programas que utilicen

	interfaces gráficos para la entrada y salida de información.

UNIDAD 10. TRABAJAR CON BASES DE DATOS RELACIONALES.	
Horas:	20
Conceptos:	Nociones básicas sobre bases de datos relacionales Conexiones a bases de datos Formas de introducción de información Métodos de recuperación de información Manipulación de la información: cambios, borrados.
Procedimientos:	Establecimiento de conexiones a una base de datos. Introducción de información. Recuperación de la información introducida. Recuperación de información calculada. Alteración de la información. Borrado de la información existente. Uso de variables vinculadas en sentencias. Uso de llamadas a procedimientos almacenados. Intercambio de datos binarios con la BD (imágenes, documentos, etc.).
Actividades de enseñanza-aprendizaje:	Establecimiento de conexiones a una base de datos previamente creada. Introducción de información desde un programa. Recuperación de la información introducida anteriormente. Recuperación de información calculada a partir de datos existentes. Alteración de la información que contiene la base de datos. Borrado de la información existente. Creación de bases de datos desde programa. Uso de conjuntos de resultados de distintos tipos. Llamadas a procedimientos almacenados. Utilizar variables vinculadas en sentencias. Utilización de varias interfaces del conector.
Contenidos:	1. INTRODUCCIÓN A MYSQL. 2. DRIVERS JDBC Y CONEXIÓN. 3. USANDO JDBC DESDE JAVA. 4. TRABAJAR CON RESULTSETS. 5. METADATOS, DATASOURCES Y ROWSETS. 6. PERSITENCIA DE OBJETOS: MAPAS, PATRÓN DAO Y JPA. 7. EJERCICIOS.
Competencias:	UC0223_3 y UC0226_3
Objetivos del ciclo:	e), j), q)

Resultados de aprendizaje:	9
Criterios de evaluación:	9.a) 9.b) 9.c) 9.d) 9.e) 9.f) 9.g) Se han identificado las características y métodos de acceso a sistemas gestores de bases de datos relacionales. Se han programado conexiones con bases de datos. Se ha escrito código para almacenar información en bases de datos. Se han creado programas para recuperar y mostrar información almacenada en bases de datos. Se han efectuado borrados y modificaciones sobre la información almacenada. Se han creado aplicaciones que ejecuten consultas sobre bases de datos. Se han creado aplicaciones para posibilitar la gestión de información presente en bases de datos relacionales.

UNIDAD 11. TRABAJAR CON BASES DE DATOS ORIENTADAS A OBJETOS.	
Horas:	17
Conceptos:	Características de las bases de datos orientadas a objetos. Creación de bases de datos. El lenguaje de consultas: sintaxis, expresiones, operadores. Recuperación, modificación y borrado de información. Tipos de datos objeto; atributos y métodos. Tipos de datos colección.
Procedimientos:	Instalación del gestor de bases de datos. Creación de bases de datos. Recuperación, modificación y borrado de información.
Actividades de enseñanza-aprendizaje:	Instalar un gestor de bases de datos orientado a objetos. Crear bases de datos. Introducir información en la base de datos. Recuperación de información de la base de datos. Borrado de información de la base de datos.
Contenidos:	1. INTRODUCCIÓN. 2. OPERACIONES BÁSICAS CON BD ORIENTADAS A OBJETOS. 3. OPERACIONES AVANZADAS CON BD ORIENTADAS A OBJETOS. 4. INTRODUCCIÓN A BD NOSQL. 5. CONEXIÓN A LA BD Y ELEMENTOS DE ALMACENAMIENTO. 6. USO DE LENGUAJE DE CONSULTA Y MODIFICACIÓN DESDE JAVA.

	7. EJERCICIOS.
Competencias:	UC0227_3
Objetivos del ciclo:	d), e), j), l)
Resultados de aprendizaje:	8
Criterios de evaluación:	8.a) 8.b) 8.c) 8.d) 8.e) 8.f) 8.g) Se han identificado las características de las bases de datos orientadas a objetos. Se ha analizado su aplicación en el desarrollo de aplicaciones mediante lenguajes orientados a objetos. Se han instalado sistemas gestores de bases de datos orientados a objetos. Se han clasificado y analizado los distintos métodos soportados por los sistemas gestores para la gestión de la información almacenada. Se han creado bases de datos y las estructuras necesarias para el almacenamiento de objetos. Se han programado aplicaciones que almacenen objetos en las bases de datos creadas. Se han realizado programas para recuperar, actualizar y eliminar objetos de las bases de datos. Se han realizado programas para almacenar y gestionar tipos de datos estructurados, compuestos y relacionados.

5 METODOLOGÍA DEL PROCESO DE ENSEÑANZA-APRENDIZAJE.

Las recomendaciones pedagógicas del módulo que aparecen en los Reales Decretos que definen el ciclo formativo dicen así: Las líneas de actuación en el proceso de enseñanza-aprendizaje que permiten alcanzar los objetivos del módulo versarán sobre:

– La interpretación y aplicación de los principios de la programación orientada a objetos.

– La evaluación, selección y utilización de herramientas y lenguajes de programación orientados a objetos.

– La utilización de las características específicas de lenguajes y entornos de programación en el desarrollo de aplicaciones informáticas.

– La identificación de las funcionalidades aportadas por los sistemas gestores de bases de datos y su incorporación a las aplicaciones desarrolladas.

– La prueba, depuración y documentación de las aplicaciones desarrolladas.

El método que se seguirá para el desarrollo del curso será el siguiente:

- Se dan pautas, consejos y técnicas y estrategias que suelen dar buenos resultados y las que no (con argumentos).
- Se facilitan materiales y recursos (apuntes, ejercicios, cuestionarios, casos prácticos, sitios web de autoaprendizaje, competiciones, etc.) no de golpe, sino dosificados a media que estos pueden ser útiles y aprovechables en el proceso de aprendizaje.
- Semanalmente se envía a cada alumno, a través de un foro de avisos, un mensaje informativo sobre ritmo de avance que se debería alcanzar durante ese período (contenidos, revisión de ejemplos, entrega de relaciones de problemas, tiempo dedicado a proyectos, realización de test, fechas previstas de controles presenciales, etc.), con el objetivo de orientar a los alumnos, que con mucha frecuencia, no tienen la posibilidad de asistir a las tutorías o conocer la evolución del curso.
- En las tutorías colectivas en primer lugar se resuelven dudas puntuales de alumnos en relación con los contenidos, ejercicios o proyectos. Después, si queda tiempo, se explican contenidos, procedimientos o técnicas utilizadas con esos elementos.
- En las tutorías individuales, si no hay solicitudes de atención individual y hay alumnos interesados en aprovechar ese tiempo, se transforman en colectivas. En caso de que se haya solicitado una atención individualizada, se atiende de forma presencial o telemática síncrona mediante Microsoft Teams.
- Los alumnos tienen a su disposición foros especializados en cada unidad donde también pueden plantear dudas o dificultades de forma pública y compartida que pueden ser resueltos por los propios alumnos, para fomentar la participación y colaboración entre ellos.
- También disponen del mail del profesor para poder comunicarse por e-mail y resolver cualquier tipo de incidencia aunque si son dudas les remito a los foros de las unidades para que puedan ser revisadas y aprovechadas por todos los alumnos del curso.
- Al principio las actividades son muy guiadas para que vayan adquiriendo conocimientos y habilidades básicas. A medida que el alumno va adquiriendo esos conocimientos se proponen actividades más abiertas, proyectos de tipo retos para que apliquen soluciones propias a problemas teniendo el objetivo como información de partida. Por las características que tiene la modalidad semipresencial, desde el inicio se utiliza el enfoque de clase invertida (se estudia en casa y en clase se resuelven dudas y se pone en práctica lo aprendido). Aunque es cierto que durante la primera evaluación sobre todo, esta solución es difícil de implementar, por lo que se realiza intermitentemente cuando es posible.
- También se utiliza material multimedia como vídeos propios o enlaces a material de terceros con demos de contenidos complejos o tareas específicas explicadas en tutorías para refuerzo de quienes asisten a las tutorías y ayuda a quien no tiene la posibilidad de hacerlo.
- Se intenta que todas las actividades tengan retroacción (soluciones correctas, respuestas de los test, posibles soluciones e controles, etc.) para fomentar el autoaprendizaje.

- Proponer actividades en grupo es complicado dada la situación de la mayoría de alumnos, así que este tipo de actividades se descarta.

6 RECURSOS.

Los principales recursos que se utilizarán:

- El aula virtual.
- Documentación y apuntes proporcionada por el profesor.
- Ejemplos y ejercicios relacionados con los contenidos de los apuntes.
- Herramientas ofimáticas y de edición de gráficos.
- Sistemas operativos libres y propietarios.
- Entornos y plataformas de desarrollo (IDE's)
- Kits de desarrollo, simulación, etc.
- Software de terceros para integrarlo en proyectos de alumnos.
- Foros de dudas para trabajar de manera colaborativa dentro de la plataforma.
- Actividades: cuestionarios interactivos, tareas de tipo ejercicios con solucionarios, prácticas a modo de proyectos de tipo reto.
- Documentación web: recursos multimedia y sitios web on-line relacionados con la materia disponibles en Internet.
- Vídeos explicativos.
- Teams para tele-aprendizaje en caso de que fuese necesario.
- Sitios web para practicar con varios enfoques: acepta el reto, codewars, etc.

7 USO DE ESPACIOS Y EQUIPAMIENTOS.

El grupo tiene un aula de referencia con capacidad para 18 puestos donde cada alumno tiene asignado un equipo informático a su disposición con las herramientas que necesita utilizar:

- Una cuenta personal en el sistema operativo con las medidas de seguridad adecuadas.
  - Herramientas ofimáticas: hojas de cálculo, procesadores de texto,
  - De desarrollo: IDE's, kits de desarrollo, compiladores, etc.
  - De comunicación: acceso a Internet, mail, aulas virtuales, etc.
  - Etc.
- El aula cuenta también con pizarra digital.
- Están organizados en forma de U y para evitar que los alumnos del fondo tenga la necesidad de girarse cada vez que se explica algo, usaremos un software de control de aula para compartir la pantalla del profesor con la de los alumnos (epoptes).
- Un aula virtual (aules) donde organizar el desarrollo del curso.
- Web familia, foros y mail del profesor por si necesitan comunicarse por algún motivo.
- Espacios en la nube donde se comparten recursos ocasionalmente.

8 MEDIDAS DE ATENCIÓN A LA DIVERSIDAD.

Considerando los principios de la Educación Inclusiva ly teniendo en cuenta también que son estudios post-obligatorios, las medidas se centran en la accesibilidad, la flexibilidad y los apoyos especializados. Se pueden implementar medidas extraordinarias individuales o bien medidas ordinarias o universales, intentando siempre que sea posible recurrir en primer lugar a las universales.

- Medidas de Respuesta de Carácter Extraordinario: Adaptación de Acceso y Evaluación:
  - Adecuación de pruebas y exámenes: Tiempo adicional, uso de soportes técnicos o materiales adaptados, lectura o transcripción de exámenes.
  - Accesibilidad Personalizada: Suministro de medios específicos o singulares (ej. material didáctico adaptado, software específico).
- Medidas de Respuesta Ordinaria y Universal: benefician en realidad a todo el alumnado, siendo el punto de partida de la inclusión:
  - Agrupamientos Flexibles: Organizar el aula con metodologías que permitan el trabajo cooperativo y el aprendizaje entre iguales.
  - Uso de Materiales Diversos: Proveer recursos en diferentes formatos (visual, auditivo, digital, impreso) y variados niveles de complejidad.
  - Apoyos y Refuerzo: Uso de horas de refuerzo educativo, actividades y material adicional de repaso, preparación y refuerzo, etc.

9 EVALUACIÓN DEL APRENDIZAJE.

La evaluación sigue los criterios establecidos en la Orden 8/2025, de 22 de abril, de la Conselleria de Educación, Cultura, Universidades y Empleo, por la que se regula la evaluación del proceso de enseñanza-aprendizaje en ciclos formativos y cursos de especialización derivados de la Ley Orgánica 3/2022, de 31 de marzo, de ordenación e integración de la Formación Profesional y el resto de la legislación vigente.

Además de ser objetiva, debe ser continua y tener como objetivo ser una herramienta más que ayude a que los alumnos alcancen los resultados de aprendizaje utilizando los criterios de evaluación como mecanismo de comprobación. Cada unidad didáctica está asociada a uno o varios resultados de aprendizaje y para que sirva a su propósito incluirá varios instrumentos diferentes tal y como recoge el PCCF de DAM en su apartado 5 “Enfoques Didácticos y Metodológicos”.

Se tendrán en cuenta también las propuestas de mejora del curso anterior donde se recogía: "Dado el aumento de entregas de ejercicios copiados o no resueltos por los propios alumnos, es interesante bajar la cantidad de ejercicios a entregar para su evaluación y bajar también el peso en la calificación en favor de métodos más objetivos".

Fases de evaluación:

- Inicial. Para conocer las características generales del alumnado y detectar problemas o circunstancias que necesiten de atención o tratamiento especial.

- Continua o formativa. Evaluar el nivel de asimilación y dominio de los resultados de aprendizaje del módulo por parte de los alumnos.
- Final o sumativa. Definir quien ha conseguido adquirir las competencias mínimas necesarias que le permiten aprobar el módulo. Tal y como establece la legislación la calificación será numérica de 1 a 10, siendo el 5 el límite que marca la superación favorable del módulo.

Instrumentos de evaluación continua:
Para evaluar al alumnado, se pueden emplear durante la evaluación continua instrumentos como:

1. Tres Controles presenciales, uno por evaluación, consistentes en resolver supuestos prácticos relacionados con:
   1.1. El objetivo es verificar los criterios de evaluación asociados a los resultados de aprendizaje que contienen los contenidos evaluados, resolviendo casos prácticos.
   1.2. Estarán formados por varios ejercicios prácticos muy concretos.
   1.3. Se realizará uno en cada evaluación.
2. Proyectos Prácticos.
   2.1. El objetivo es entrenar el análisis de problemas, la elaboración de estrategias y enfoques de solución, poner en práctica el uso de elementos de los lenguajes y las habilidades para implementar la solución de casos reales simulados con cierta complejidad para poner en práctica los criterios de evaluación.
   2.2. En función de la complejidad, a veces serán guiadas y otras veces se piden soluciones abiertas. Opcionalmente (a veces) aparecerán problemas adicionales de mayor complejidad para quienes quieran practicar más. De no hacerlos, no habrá penalización puesto que tendrán carácter voluntario. Si se entregan, puede obtener más puntos extra en la parte de prácticas.
   2.3. Como instrumento de evaluación se utilizarán rúbricas de cada ejercicio.
3. Controles de teoría y ejercicios prácticos de tipo de test:
   3.1. Unas pruebas con límite de tiempo donde hay que contestar 10 preguntas (teórico-prácticas) elegidas al azar de entre un bloque de preguntas previamente preparadas relacionadas con conceptos y elementos de cada unidad.
   3.2. El objetivo es que los alumnos intenten llevar los contenidos al día y no se desconecten de la materia al mismo tiempo que se evalúa como se han asimilado los conceptos y elementos de cada unidad.
   3.3. Los realizarán de forma autónoma y tendrán retroalimentación inmediata del resultado.
4. Problemas:
   4.1. Unos ejercicios que ellos mismos se corregirán.
   4.2. Se facilitarán posibles soluciones.
   4.3. Se podrán plantear soluciones alternativas y evaluar pros y contras de cada una.

4.4. El objetivo es que los alumnos no se desconecten de la asignatura durante un tiempo excesivo y que puedan detectar aspectos que no acaban de entender o no se hayan comprendido correctamente.

Criterios de cualificación en la evaluación continua:

- Habrá 3 evaluaciones durante el curso.
- 1ª y 2ª evaluación la nota se obtendrá única y exclusivamente del examen final de la evaluación
- 3ª La nota se obtendrá de un 50% trabajo final y 50% del examen de la evaluación.
- La nota final será la media aritmética de las 3 evaluaciones. Siempre que en cada evaluación se haya obtenido una nota mínima de 4.5 para poder mediar

Criterios de calificación en las convocatorias finales:

Evaluación ordinaria:

- Se realizará un examen de los RAs que tenga pendientes de superar.

Evaluación extraordinaria:

- Se aplica un criterio similar al de la convocatoria ordinaria. Aquellos alumnos con una nota media inferior a 5 (no ha conseguido los RA asociados a las unidades) tras realizar la convocatoria ordinaria, tendrán que recuperar solamente los resultados de aprendizaje aún no conseguidos ni en la evaluación continua ni en la evaluación ordinaria.

Ponderación de la nota:

En principio con el objetivo de que la evaluación sea variada, continuada y objetiva se pretende valorar las actividades de los alumnos mediante esta ponderación:

- Examen teórico/práctica → 100% en 1ª y 2ª evaluación. 50% en la 3ª
- Trabajo final → 50% en la tercera evaluación

En el caso de detectar que de manera masiva las actividades prácticas no son realizadas por los propios alumnos, estas actividades pierden su esencia y en este caso no tiene sentido usarlas como herramienta de evaluación porque carecen de la objetividad necesaria y deja de ser útil en el proceso de aprendizaje. Se informaría a los alumnos del cambio de método de ponderación y la calificación recaería exclusivamente en los exámenes. Aunque se pueden seguir proponiendo el resto de las actividades, ya no se usarían como herramientas de evaluación.

Nota final del módulo:

La misma fórmula aplicada a todas las actividades realizadas durante el curso. En caso de realizar recuperaciones en convocatoria ordinaria o extraordinaria, se sustituirá la nota del control de las unidades recuperadas (es decir, sus RA asociadas) por la mejor de las notas obtenidas. En caso de recuperar toda la materia, la nota obtenida en el control será la nota final del módulo.

Aprobado >= 5

10 ACTIVIDADES COMPLEMENTARIAS Y EXTRAESCOLARES

En esta modalidad es imposible garantizar la asistencia de alumnos a actividades complementarias o extraescolares, por tanto, no está previsto realizar ninguna, aunque si surge alguna interesante se informará de su realización a los alumnos para el caso de que puedan asistir.

11 CRITERIOS Y PROCEDIMIENTOS PARA LA EVALUACIÓN DEL DESARROLLO DE LA PROGRAMACIÓN Y DE LA PRÁCTICA DOCENTE, ASÍ COMO LOS CRITERIOS DE CALIFICACIÓN.

Para medir el desempeño de la práctica docente será necesario recolectar información que permita elaborar y definir indicadores medibles para juzgar la calidad de este proceso en función de los objetivos iniciales propuestos:

- Datos cuantitativos de éxito en los alumnos: tasa de aprobados por evaluación, calificación obtenida en actividades o productos elaborados por los alumnos, porcentaje de entregas, participación, aprobados en convocatoria ordinaria/extraordinaria.
- Ajuste Temporal: Grado de cumplimiento del calendario y la secuencia de unidades didácticas y actividades realizadas.
- Idoneidad Metodológica: Valoración de la efectividad de las metodologías y los recursos didácticos utilizados (si determinadas actividades facilitan o no el aprendizaje).
- Resultados de la Detección de Barreras: Análisis de si las medidas de atención a la diversidad han sido suficientes para reducir barreras de aprendizaje.

Los procedimientos para la Evaluación de la Práctica Docente:
Recogida de Datos mediante:

- Observación periódica por unidad didáctica impartida: problemas de ritmo, motivación o gestión del aula.
- Retroacción de alumnos: consultas sobre las dificultades que encuentran, sugerencias que puedan tener, valoración de las actividades, opinión de los

estudiantes sobre la claridad de las explicaciones, la utilidad de los recursos y la dificultad de la evaluación.
- Análisis de resultados académicos: Estudio comparativo de calificaciones y tasas de abandono/resultados académicos.
- Coordinación Docente: Actas y acuerdos del departamento o del equipo docente, intercambio de estrategias, acciones o enfoques para resolver posibles detectados.

Objetivos:
- Mejorar los resultados para maximizar el aprendizaje de los alumnos...
- Anticiparse a los problemas antes de que aparezcan o al menos...
- Detectarlos y poder actuar tan rápido como sea posible.

Criterios de Calificación del Alumnado.
Ponderación por Resultados de Aprendizaje a través de los RA: cada unidad está asociada a varios RA. Un mismo RA puede estar asociado a diferentes unidades. En realidad, dadas las características técnicas del módulo, las unidades posteriores siguen utilizando los RA de las unidades anteriores. Especificar un peso porcentual de cada RA en la calificación final del módulo no tiene mucho sentido por este hecho: no son independientes y todos tienen una importancia similar en el aprendizaje dado que para alcanzar uno necesitas utilizar los anteriores.

Resultado de Aprendizaje (RA)	Ponderación (%)
RA 1 al RA 9	Tienen el mismo peso.

Entre el 10 y el 15% de los criterios de evaluación pueden recaer en la empresa. Estos criterios pueden repercutir favorable o desfavorablemente en la calificación porcentualmente.

Requisitos Mínimos:
La nota mínima para superar el módulo (es 5 sobre 10). Hay 3 RA fundamentales que son las sentencias de control, la programación estructurada y la orientación a objetos. No es obligatorio superar todos los RA de forma independiente, aunque no superar estos, dada la cantidad de criterios que contienen, condicionará no aprobar el módulo, aunque se ponderen igual que el resto (que para aprobarlos es necesario dominar los mencionados). Se realizará una media de todos los criterios de evaluación.

Si por evaluación continua no se alcanza un 5 de nota o el alumno no ha participado en esta fase, el alumno puede presentarse a la convocatoria del examen final ordinario para evaluarse de todos los RA. Aquellos que hayan realizado la evaluación continua y tengan algunos RA superados y otros pendientes, solo se evaluarán de los que tengan pendientes.

En el final ordinario, solo se examinará de las unidades (resultados de aprendizaje asociados a la unidad) pendientes de superar.

Si no se supera el módulo en la convocatoria ordinaria, para la convocatoria extraordinaria se eliminan las unidades superadas (RA asociados) en la convocatoria ordinaria o en la evaluación continua.

12 PLAN DE RECUPERACIÓN.

El plan de recuperación para los alumnos repetidores de cursos anteriores consiste en:

- Estos alumnos pueden seguir la evaluación continua igual que el resto de los alumnos.
- El alumno repetidor tendrá la posibilidad de realizar la evaluación continua o si lo prefiere recuperar los RA asociados a las unidades impartidas en cada evaluación presentándose exclusivamente a los controles de dichos contenidos donde se evalúa de los RA pendientes de superar sin realizar ni tener en cuenta ninguna otra actividad formativa.
- También pueden elegir presentarse directamente a las convocatorias ordinaria o extraordinaria para evaluarse.

En caso de no seguir la evaluación continua, se usará el criterio de comprobar el nivel de consecución de los RA aprobados previamente y el obtenido en los pendientes de conseguir de los cuales se examinará mediante una prueba práctica objetiva.

Cuando comience el período de formación en la empresa, se ofrecerá a los alumnos con el módulo pendiente de aprobar, un plan de repaso de aproximadamente 26 sesiones (el 10% de las horas del módulo) horas que consistirá en:

- Repaso de conceptos.
- resolución de dudas.
- Realizar ejercicios.
- Completar prácticas.

El objetivo es repasar los resultados de aprendizaje pendientes y facilitar su consecución.

13 PLAN DE FOMENTO DE LA LECTURA.

Se propondrá la lectura de documentación de varios foros relacionados con la programación y artículos, sitios, series de televisión, películas, libros, literatura y todo elemento que pueda favorecer la asimilación o ampliación de conocimientos y la mejora de habilidades personales y profesionales.

14 PROYECTO INTERMODULAR.

El resultado del aprendizaje etiquetado como RA 2 «Escribe y prueba programas sencillos, reconociendo y aplicando los fundamentos de la programación orientada a objetos.» se trabajará también en el proyecto intermodular.